2026-06-10-verifier-discovery-wiring-design

Verifier Discovery → Catalog Wiring Design

Revision: 1 **Last modified:** 2026-06-10 **Maintainer:** design subagent (read-only) **Status:** DESIGN — no code changed; this doc only specifies the wiring.

0. Scope & ground truth

SP2 added the unified exposure catalog (`GET /v1/catalog`) joining the AI-debate ensemble (+presets), HelixLLM, every discovered provider, and every VERIFIED model. It is **honest-empty of verified-model entries today** because the `VerifiedModelSource` is constructed over a `nil` discovery service.

Exact wiring point (the prompt said line 1295 — that is wrong; 1295 is the discovery *handler*, a different thing). The real call is:

`submodules/helix_agent/internal/router/router.go:779`

```
Verified: catalog.NewDiscoveryVerifiedSource(nil), // discovery
        svc not wired here → honest-empty
```

All `file:line` citations below are verified against the tree at the time of writing.

---

1. How `verifier.ModelDiscoveryService` is built + populated (Task 1)

1.1 Constructor & dependencies

`submodules/helix_agent/internal/verifier/discovery.go:82`

```
func NewModelDiscoveryService(
    vs *VerificationService,    // discovery.go:24 field
        verificationService
    ss *ScoringService,         // discovery.go:25 field
        scoringService
    hs *HealthService,          // discovery.go:26 field
        healthService
    cfg *DiscoveryConfig,       // discovery.go:27 field config
) *ModelDiscoveryService
```

- `cfg == nil` → `DefaultDiscoveryConfig()` (discovery.go:88-90).
- Builds an internal `http.Client{Timeout: 30s}` (discovery.go:99-101) and a `stopCh` (discovery.go:102).

1.2 What it needs to actually populate

It is **NOT** a thin client of a central LLMsVerifier service. Its `runDiscoveryPipeline` (discovery.go:151) hits **each provider's own `/models` endpoint directly**: - `getDiscoveryEndpoint` (discovery.go:241-266) hard-maps provider name → upstream URL (`openai → https://api.openai.com/v1/models`, `ollama → http://localhost:11434/api/tags`, etc.) or `cred.BaseURL + "/models"`. - It therefore requires `[]ProviderCredentials` (discovery.go:75-79: `ProviderName`, `APIKey`, `BaseURL`) passed to `Start(credentials)`.

1.3 The poll loop

`Start([]ProviderCredentials)` (discovery.go:107) launches `discoveryLoop` (discovery.go:131): an **initial** `runDiscoveryPipeline`, then a `time.NewTicker(s.config.DiscoveryInterval)` loop (discovery.go:137-147). - Pipeline = discover (discovery.go:155) → verify (discovery.go:158) → score → select. - `GetDiscoveredModels()` (discovery.go:452) returns `[]*DiscoveredModel`; the catalog adapter reads `Verified`, `Provider`, `ModelID`, `OverallScore` (registry_adapter.go:60-78).

1.4 Config / interval defaults — CONST-038 conflict to fix

`DefaultDiscoveryConfig()` sets `DiscoveryInterval: 24 * time.Hour` (discovery.go:510). **CONST-038 requires ≤ 60 s poll**. The wiring MUST override the interval to ≤ 60 s (env-driven, default 60 s) — see §4.3. (CONST-037's "verified within 24h" is the *staleness budget* for a model, not the *poll* interval; the 24 h default conflates them and breaks CONST-038.)

---

2. Where the router builds its dependencies (Task 2)

Router build entry: `SetupRouterWithContext(cfg)` (`router.go:158`). Relevant in-scope objects at the `:779` catalog block:

- `providerRegistry := services.NewProviderRegistry(...)` — router.go:263, stored on `rc.ProviderRegistry` (router.go:264) "Exposed for StartupVerifier integration".
- `providerRegistry.GetStartupVerifier()` — provider_registry.go:583, returns `*verifier.StartupVerifier`. Already consulted at router.go:907 / :927 for the debate team + intent router.
- `providerRegistry.GetDiscovery()` — provider_registry.go:532, returns `*services.ProviderDiscovery` (a *provider* discovery, not the model-discovery service — do not confuse).
- `logger`, `cfg`, `os.Getenv(...)` are all in scope at the catalog block (router.go:775-786).

So the catalog block at `:773-786` already has `providerRegistry` (it uses it for `NewRegistryProviderSource`). **Everything needed to obtain a real verified source is reachable here.**

---

3. WHY it's `nil` today (Task 3)

3.1 Mechanically

`NewModelDiscoveryService` is **never constructed in any production path** — grep across the submodule finds it ONLY in test files (`internal/verifier/discovery_test.go`, `internal/handlers/discovery_handler_test.go`). Confirmed: no non-test caller exists. SP2 therefore had nothing populated to pass and correctly chose honest-empty (registry_adapter.go:50-58 returns `nil` source for `nil` disc → catalog.go:218 skips the verified-model section) rather than fabricating a list (anti-bluff §11.4 / CONST-036).

3.2 The architectural reason — two parallel discovery pipelines

helix_agent has **two** independent verification/discovery mechanisms:

	`ModelDiscoveryService` **(discovery.go)**	`StartupVerifier` **(startup.go)**
Constructed in prod?	**No** (tests only)	**Yes** — built inside `NewProviderRegistry`, reachable `providerRegistry.GetStartupVerifier()` (provider_registry.go:583)
Pipeline	discover→verify→score→select, polled (discovery.go:131)	discover→verify→subscriptions→score→rank→selectDel (startup.go:218-274)
Output type	`[]*DiscoveredModel` w/ `Verified/OverallScore` (discovery.go:452)	`[]*UnifiedProvider`, each w/ `Verified bool`, `S Models []UnifiedModel` (provider_types.go:74,76,8
Verified accessor	`GetDiscoveredModels()`	`GetVerifiedProviders()` (startup.go:1638) / `GetRankedProviders()` (startup.go:1611)

`catalog.NewDiscoveryVerifiedSource` is typed to the *unconstructed* `ModelDiscoveryService`. The **real, already-running, already-populated** verified data lives in `StartupVerifier`. So today's `nil` is not just "forgot to pass it" — it is "the type the catalog wants is the one nobody builds."

3.3 Is a constructed discovery service available elsewhere?

No `ModelDiscoveryService` is. But a constructed, populated **verified-model source** IS available: `StartupVerifier` via `providerRegistry`. This drives the recommended wiring (§4, Option B).

---

4. Wiring design (Task 4)

Two options. **Option B is recommended** (reuses the real populated pipeline; no duplicate polling; satisfies CONST-036/037/038 with the data that already drives the debate team). Option A is documented for completeness / future use.

4.1 Option A — construct a real `ModelDiscoveryService` (matches the existing adapter type)

1. In `SetupRouterWithContext`, after `providerRegistry` is built, construct:

```
vs := verifier.NewVerificationService(verifier.DefaultConfig())
ss, _ := verifier.NewScoringService(verifier.DefaultConfig())
hs := verifier.NewHealthService(...)            // confirm
        constructor
dcfg := verifier.DefaultDiscoveryConfig()
dcfg.DiscoveryInterval = pollInterval()         // ≤ 60s,
        CONST-038
disc := verifier.NewModelDiscoveryService(vs, ss, hs, dcfg)
disc.Start(buildCredentialsFromRegistry(providerRegistry)) //
        []ProviderCredentials
rc.modelDiscovery = disc                         // for
        graceful Stop()
```

2. Pass it at router.go:779:

```
Verified: catalog.NewDiscoveryVerifiedSource(disc),
```

3. Add `disc.Stop()` to `RouterContext.Shutdown()` (router.go:74).

Cost: a SECOND verify/score pipeline (the StartupVerifier already does this), double the upstream `/models` calls, and you must derive `[]ProviderCredentials` from the registry. Higher blast radius, duplicates work.

4.2 Option B — adapt `StartupVerifier` (RECOMMENDED)

Add a second `VerifiedModelSource` implementation in the catalog adapter that reads the already-populated `StartupVerifier`. No new poller; reuse the live one.

New constructor + adapter in `internal/catalog/registry_adapter.go` (alongside the existing `discoveryVerifiedSource`):

```
// NewStartupVerifierSource builds a VerifiedModelSource over the
        live
// StartupVerifier (the pipeline that already drives the debate
        team).
func NewStartupVerifierSource(sv *verifier.StartupVerifier)
        VerifiedModelSource {
    if sv == nil { return nil } // honest-empty preserved
    return &startupVerifierSource{sv: sv}
}

func (s *startupVerifierSource) VerifiedModels() []VerifiedModel
        {
    if s == nil || s.sv == nil { return nil }
    var out []VerifiedModel
```

```
    for _, p := range s.sv.GetVerifiedProviders() {      //
        startup.go:1638
        if !p.Verified { continue }
        for _, m := range p.Models {                     //
        provider_types.go:89
            if !m.Verified { continue }                  // model-
        level Verified, provider_types.go (UnifiedModel.Verified)
            // CONST-037 staleness gate: skip models whose
        VerifiedAt is older
            // than 24h (provider_types.go
        UnifiedModel.VerifiedAt).
            if time.Since(m.VerifiedAt) > 24*time.Hour {
        continue }
            out = append(out, VerifiedModel{
                Provider:     p.Type,                     //
        normalize lower-case in catalog.Build (catalog.go:223)
                ModelID:      m.ID,
                Verified:     true,
                OverallScore: m.Score,
            })
        }
    }
    return out
}
```

Wire at router.go:779:

```
Verified:
        catalog.NewStartupVerifierSource(providerRegistry.GetStartupVerifi
```

`GetStartupVerifier()` may return `nil` before the verification pipeline has run (router.go:917/:936 already handle the nil case) → adapter returns `nil` source → catalog stays honest-empty until verification completes. This is correct: it never fabricates, and self-heals once `VerifyAllProviders` finishes (startup.go:218).

4.3 `/v1/providers` path (Task 4, providers half)

`/v1/providers` (router.go:792-) currently emits each provider's `capabilities.SupportedModels` (self-declared display hint), NOT verifier-backed models. To show **real Verified models** there too, augment the handler to read `providerRegistry.GetStartupVerifier().GetVerifiedProviders()` and emit a `verified_models: []` field per provider (provider `Type` + each `m.ID/m.Score/m.VerifiedAt` where `m.Verified && fresh`). Keep `supported_models` as the display hint; add `verified_models` as the verifier-authoritative list (CONST-036: verifier is the source of truth; the self-declared list must never be promoted to "verified").

4.4 Config / env it needs

- `pollInterval()` — env `HELIX_VERIFIER_DISCOVERY_INTERVAL` (default `60s`, clamp to ≤ 60 s per CONST-038). Option B reuses the StartupVerifier's own re-verify cadence; if that cadence is > 60 s, add a periodic re-verify trigger ≤ 60 s OR document that

the StartupVerifier re-verify loop already satisfies it (verify before claiming — do not assume).
- Verified-staleness budget — fixed 24 h per CONST-037 (`m.VerifiedAt`), applied in the adapter (§4.2).
- No central "LLMsVerifier URL" is needed for Option B (the StartupVerifier discovers from env-configured providers). For Option A, the per-provider URLs are hard-mapped in `getDiscoveryEndpoint` (discovery.go:244-259); only `cred.BaseURL` overrides apply.

---

5. RED-first test plan (Task 5)

Goal: a test that proves `/v1/catalog` returns real verified models when a (test) verified source is wired, and **FAILS today** (nil → empty), per §11.4.115 (RED-on-broken-artifact + `RED_MODE` polarity) and §11.4.43.

5.1 Pure unit RED (no infra) — the primary guard

Catalog service is already unit-testable (catalog_test.go exists). Add a test in `internal/catalog/`:

```
// fakeVerifiedSource returns one Verified model — anti-bluff: a
        concrete,
// asserted value, not a mock that returns whatever the test
        wants.
type fakeVerifiedSource struct{}
func (fakeVerifiedSource) VerifiedModels()
        []catalog.VerifiedModel {
    return []catalog.VerifiedModel{{Provider: "openai", ModelID:
        "gpt-4o",
        Verified: true, OverallScore: 0.91}}
}

func TestCatalog_VerifiedModelsFlow_RED(t *testing.T) {
    redMode := os.Getenv("RED_MODE") != "0" // default 1 =
        reproduce defect
    var verified catalog.VerifiedModelSource
    if redMode {
        verified = catalog.NewDiscoveryVerifiedSource(nil) //
        TODAY's wiring → nil
    } else {
        verified = fakeVerifiedSource{}                    //
        POST-fix wiring
    }
    svc := catalog.New(catalog.Options{
        Providers: catalog.NewRegistryProviderSource(reg),
        Verified:  verified,
    })
    entries := svc.Build()
    got := countKind(entries, catalog.KindModel) // helper
```

```
    if redMode {
        // RED asserts the DEFECT is present: zero verified
        models today.
        require.Zero(t, got,
        "RED: expected honest-empty with nil source")
    } else {
        require.GreaterOrEqual(t, got, 1, "GREEN: verified model
        must appear")
        require.Contains(t, names(entries), "openai/gpt-4o")
    }
}
```

- `RED_MODE=1` (default): passes today, capturing the defect baseline (catalog is empty of models with the current `nil` wiring) — the proof the guard is real.
- `RED_MODE=0`: fails today (no fake wired in production), passes after the §4 wiring lands. This is the polarity-switch guard (§11.4.115); register it as a standing regression guard per §11.4.135.

5.2 HTTP-level RED (handler)

Spin the catalog handler (`catalog.NewHandler`, handler.go) on an httptest server, `GET /catalog`, assert the JSON body's model-kind count under the same `RED_MODE` polarity. Catches the wiring at the route layer (router.go:782).

5.3 Adapter RED for Option B

A focused test for `NewStartupVerifierSource`: feed a `*StartupVerifier` whose `GetVerifiedProviders()` returns a provider with one fresh `Verified` model and one stale (`VerifiedAt` > 24h) model; assert only the fresh one surfaces (CONST-037 gate) and a non-verified model is excluded. Paired §1.1 mutation: drop the `time.Since(...) > 24h` guard → the stale model leaks → test FAILs.

---

6. Honest boundary: unit vs integration (Task 6)

- **The wiring CAN and SHOULD be unit-tested with a fake `VerifiedModelSource`.** `catalog.VerifiedModelSource` is a single-method interface (catalog.go:104-106). A test fake satisfies it with zero infra. The RED→GREEN flip in §5.1/§5.2 needs **no** running LLMsVerifier, no provider keys, no DB — it proves the *catalog-side contract* (nil → empty; non-nil populated → models appear). Mocks/fakes are permitted here because these are unit tests (CONST-050(A)).

- **An end-to-end "real verified models actually flow" claim REQUIRES integration infra** (CONST-050(B), Rule 5): the StartupVerifier pipeline only populates `GetVerifiedProviders()` after `VerifyAllProviders` runs against **real provider endpoints with real API keys** (startup.go:218, getDiscoveryEndpoint reaches live `https://api.openai.com/v1/models` etc.). So:

    - Unit layer (fake source) — proves the wiring contract. No infra. Primary RED guard.
    - Integration layer — boot helix_agent with at least one real provider key (e.g. an Ollama at `localhost:11434`, the one no-cloud-key option in `getDiscoveryEndpoint`, discovery.go:253), let startup verification run, then

`GET /v1/catalog` and assert ≥1 `verified` model entry whose `Provider/Model` matches a really-verified Ollama model — captured runtime evidence per §11.4.5/§11.4.69 (`network_connectivity` / real HTTP body), NOT metadata-only.
    - **Ollama is the cheapest real-infra path** (local, no secret); cloud providers need keys from `.env` (CONST-042 — never committed).

---

7. Summary

- **Why nil today:** `catalog.NewDiscoveryVerifiedSource` is typed to `*verifier.ModelDiscoveryService`, which is **constructed only in tests, never in production** (grep-confirmed). SP2 passed `nil` and chose honest-empty (registry_adapter.go:54) rather than fabricate (anti-bluff). Compounding it, the real, populated verified data lives in a **different** object — `StartupVerifier` (`providerRegistry.GetStartupVerifier()`, startup.go:1638) — not in `ModelDiscoveryService`.
- **The wiring point:** `submodules/helix_agent/internal/router/router.go:779` (the `Verified:` field of `catalog.Options`). `providerRegistry` is already in scope there. **Recommended (Option B):** add a `StartupVerifier`-backed `VerifiedModelSource` adapter in `internal/catalog/registry_adapter.go` and pass `catalog.NewStartupVerifierSource(providerRegistry.GetStartupVerifier())` — reuses the live pipeline, no duplicate poller, applies the CONST-037 24h staleness gate in the adapter and CONST-038 ≤60s cadence via the existing re-verify loop (verify the loop period before claiming compliance). Mirror the same source into `/v1/providers` as a `verified_models` field.
- **Test boundary:** the wiring contract (nil→empty vs populated→models) is fully **unit-testable with a fake** `VerifiedModelSource` (single-method interface, no infra) — this is the primary RED guard with a `RED_MODE` polarity switch. The "real models really flow" claim is **integration-only** (StartupVerifier needs real provider endpoints + keys; Ollama on `localhost:11434` is the cheapest no-secret real path) with captured runtime evidence.